EXPENSER

Personal Finance Tracker

A Django-based Web Application for Income, Expense & Budget Management

---

Submitted By

Student Name	Student ID
Shihab Shaharia	2023100010136
Abid Ahmed Angkon	2023200010039
Mahfuz Iqram Sunny	2023200010034

PROJECT REPORT

This Report Presented in Partial Fulfillment of the Course

CSE474.3: Software Development and Project Management

in the Computer Science and Engineering Department

Submitted To: Md. Anower Perves (AP)

Lecturer, Department of CSE, Southeast University

SOUTHEAST UNIVERSITY

Dhaka, Bangladesh

June 6, 2026

Contents

Abstract

This report documents the design, development, and implementation of **Expenser**, a web-based personal finance management application built as part of the CSE474.3 Software Development and Project Management Lab at Southeast University.

Expenser allows registered users to record daily income and expense transactions, organise them by category and tag, set monthly spending budgets per category, and generate financial reports over flexible date ranges. Reports can also be exported as CSV files. The application is built using Python 3.12 and Django 5.2 on the backend, PostgreSQL 16 as the database, and Tailwind CSS for the frontend. It runs on a Linux virtual machine environment.

The project was developed by a team of three students using an Agile methodology. Work was split across two sprints: the first sprint covered the initial development of all features, and the second sprint focused on testing, bug fixing, and refining the application based on what we found after running everything together for the first time.

The source code is publicly available at:

https://github.com/shihabshaharia/expenser

Chapter 1

Introduction

Expenser is a web-based personal finance management application developed as part of the CSE474.3 Software Development and Project Management Lab course at Southeast University. The main idea behind this project was to build something useful while learning Django properly. We wanted to make a simple tool where a user can log their daily income and expenses, keep track of monthly budgets, and see reports on how they are spending.

The application is built using Python and Django on the backend, PostgreSQL as the database, and Tailwind CSS for the frontend styling. It runs on a Linux virtual machine environment during development.

1.1 Project Goals

- Allow users to record income and expense entries with categories and tags
- Let users set monthly budgets per category and track spending
- Generate financial reports over different time periods
- Export report data as CSV
- Implement Django's authentication system for secure user accounts
- Practice team-based development using Git and GitHub

Chapter 2

Project Planning

2.1 Development Methodology — Agile

For this project we chose an **Agile** development approach rather than Waterfall. The main reason is that with three people working in parallel, we could not afford to wait for one phase to fully finish before starting the next. Requirements also changed slightly as we went — for example, the dashboard layout was redesigned after we saw how the data actually looked, and the entry form was updated to filter categories by type after we tested it. These kinds of mid-development adjustments are exactly what Agile is built for.

With a Waterfall approach we would have had to plan every single detail before writing any code, which is unrealistic for a university group project where everyone is learning the framework at the same time.

2.1.1 Why Agile Suited This Project

- We had three developers working on separate features simultaneously, which maps naturally to Agile's parallel workstream model.
- Features were developed, merged, and tested incrementally rather than all at once at the end.
- When something was not working as expected (for example, template rendering bugs or form validation issues), we could fix it in the next iteration without it blocking everyone else.
- The project scope was roughly defined at the start, but the exact implementation details evolved as we built and tested things.

2.1.2 Key Agile Principles We Applied

- **Iterative delivery** — We delivered working features at the end of each sprint rather than waiting until the very end.

- **Continuous integration** — Each person worked on a named branch and submitted a pull request when their feature was ready. The main branch always had working code.
- **Responding to change** — When we found issues during testing (like the budget currency showing the wrong symbol, or the category filter not working), we went back and fixed them in Sprint 2 rather than leaving them.
- **Team collaboration** — Decisions about design and implementation were made as a group via WhatsApp and quick meetings, not handed down by one person.

2.2 Sprint Plan

We organised the project into two sprints. The first sprint covered all the initial development. The second sprint was focused on testing, fixing problems we found, and refining the application based on what we saw after running it for the first time.

2.2.1 Sprint 1 — Initial Development (June 1–3, 2026)

The goal of Sprint 1 was to get a fully working first version of the application. All three team members worked on their assigned modules in parallel on separate GitHub branches.

Who	Tasks completed
Shihab	Django project setup, PostgreSQL config, all database models and migrations, Django admin, custom User model, registration and login views, post-save signal for category seeding, base HTML templates, dashboard view with ORM aggregations
Abid	Entry CRUD views and EntryForm with validation, Category CRUD views, Budget CRUD views and BudgetForm, entry list filtering by type/category/tag/date, pagination, process_recurring management command
Mahfuz	ReportView with date-range logic, CSV export view, custom template tag library (currency filter, percentage filter, tag_pills inclusion tag, category colour filter), Chart.js area chart and donut chart in the report template

Table 2.1: Sprint 1 deliverables

At the end of Sprint 1, each person submitted a pull request to the main branch. These were reviewed and merged on June 4, 2026. After merging, we ran the full application together for the first time and noted a list of things to fix.

2.2.2 Sprint 2 — Testing, Fixing, and Refining (June 4–6, 2026)

Sprint 2 is a good example of Agile's iterative nature. After merging everyone's work in Sprint 1, we tested the application properly and found several issues. These were not things we could have caught earlier because they only appeared when all the parts were running together.

Issues found and fixed in Sprint 2:

- **Template rendering errors** on the dashboard and reports pages caused by multi-line Django comment syntax that does not support spanning across lines. Fixed by converting all `{# ... #}` comments to `{% comment %}...{% endcomment %}` blocks.
- **Currency symbol** was showing as GBP (£) instead of BDT (Tk.). Updated the currency filter and all Chart.js tooltip callbacks.
- **Budget admin** had a Python syntax error from a JavaScript-style `//` comment accidentally left in the file. Fixed.
- **Dashboard charts** were not rendering side by side on the page. Fixed by replacing compiled Tailwind grid classes with inline CSS grid styles.
- **Category colour avatars** were invisible on the budget and entry list pages because the dynamic Tailwind class names were not included in the compiled CSS output. Fixed by adding a `category_color_hex` filter that returns a hex colour value for use in inline styles.
- **Entry form** category dropdown was not filtering by type. Added JavaScript to show only income categories when Income is selected and only expense categories when Expense is selected.
- **Merge conflicts** between branches were resolved collaboratively, keeping the best version of each change.

This back-and-forth between building and fixing is one of the main strengths of Agile. In a Waterfall project, these bugs would only have been discovered at the very end during a final testing phase, which would have left very little time to fix them. Because we were testing after each sprint, we had time to go back and make things right.

2.3 Task Distribution

Each team member owned a clear part of the codebase, which reduced conflicts and made parallel development easier:

Who	Sprint 2 work
Shihab	Fixed template comment bugs across all pages, redesigned dashboard layout, added sparkline charts to stat cards, rebuilt login and register pages as standalone templates
Abid	Fixed budget admin syntax error, updated entry form with client-side category filtering, wired drawer system for category and budget forms
Mahfuz	Fixed currency symbol in template tags and chart callbacks, fixed category colour hex filter, resolved template tag errors in report page

Table 2.2: Sprint 2 deliverables

- **Shihab Shaharia** — Project structure, models, authentication, admin, dashboard, all HTML templates and UI
- **Abid Ahmed Angkon** — All CRUD views and forms for entries, categories, and budgets
- **Mahfuz Iqram Sunny** — Reports, CSV export, custom template tags, Chart.js visualisations

2.4 Tools Used

- **GitHub** — version control, branching, pull requests, code review
- **WhatsApp** — quick team communication and coordination
- **Google Docs** — shared planning notes and task checklist
- **draw.io** — initial ER diagram sketch before implementing models

Chapter 3

Environment Setup (Linux VM)

The project runs on a Linux virtual machine (Ubuntu 22.04). Below are the steps to set it up from scratch.

3.1 System Requirements

- Ubuntu 22.04 LTS (or similar Debian-based Linux)
- Python 3.11 or 3.12
- PostgreSQL 14 or later
- Node.js 18 or 20 LTS (for Tailwind CSS build)
- Git

3.2 Step-by-Step Setup

3.2.1 1. Install System Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv
sudo apt install -y postgresql postgresql-contrib
sudo apt install -y nodejs npm git
```

3.2.2 2. Set Up PostgreSQL

```
sudo -u postgres psql
```

Inside the PostgreSQL shell:

```
CREATE DATABASE expenser_db;
CREATE USER expenser_user WITH PASSWORD 'expenser_pass';
GRANT ALL PRIVILEGES ON DATABASE expenser_db TO expenser_user;
\q
```

3.2.3 3. Clone the Repository

```
git clone https://github.com/shihabshaharia/expenser.git
cd expenser
```

3.2.4 4. Create Virtual Environment and Install Dependencies

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

3.2.5 5. Configure Environment Variables

Create a `.env` file in the project root:

```
DJANGO_SECRET_KEY=your-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=expenser_db
DB_USER=expenser_user
DB_PASSWORD=expenser_pass
DB_HOST=localhost
DB_PORT=5432
```

3.2.6 6. Run Migrations and Seed Data

```
python manage.py migrate
python manage.py createsuperuser
python manage.py seed_categories
```

3.2.7 7. Build Tailwind CSS and Start the Server

```
python manage.py tailwind build
python manage.py runserver
```

The application will be available at `http://127.0.0.1:8000`.

Chapter 4

Project Structure

4.1 File Structure

The project follows Django's standard app-based layout. Each feature has its own Django app:

```
expenser/                    <- Django project config
    settings.py
    urls.py
    wsgi.py

accounts/                    <- User registration, login, signals
    models.py
    views.py
    urls.py
    signals.py
    forms.py

core/                        <- Categories, Tags, Entries, Dashboard
    models.py
    views.py
    forms.py
    urls.py
    dashboard_urls.py
    templatetags/
        expenser_tags.py     <- Custom filters and template tags
    management/commands/
        seed_categories.py
        process_recurring.py

budgets/                     <- Budget model and CRUD
    models.py
    views.py
    forms.py
    urls.py
```

```
reports/                    <- Reports and CSV export
    views.py
    urls.py

templates/                  <- All HTML templates
    base.html
    core/
    budgets/
    reports/
    registration/
    accounts/

theme/                      <- Tailwind CSS build pipeline
    static_src/
        tailwind.config.js
```

4.2 Key Configuration Files

4.2.1 settings.py

The settings file controls how Django is configured. Some important sections:

```
INSTALLED_APPS = [
    # Django built-ins
    'django.contrib.admin',
    'django.contrib.auth',
    ...
    # Third party
    'tailwind',
    # Our apps
    'theme', 'accounts', 'core', 'budgets', 'reports',
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME'),
        'USER': os.environ.get('DB_USER'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
```

```
}

AUTH_USER_MODEL = 'accounts.User'

LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/accounts/login/'
```

All sensitive values (secret key, database password) are loaded from a `.env` file using `python-dotenv` so they are not hardcoded in the source code.

4.2.2 URL Routing

Django routes HTTP requests using `urls.py` files. The top-level URL config in `expenser/urls.py` distributes requests to each app:

```
urlpatterns = [
    path('admin/',    admin.site.urls),
    path('accounts/', include('accounts.urls')),
    path('entries/',  include('core.urls')),
    path('budgets/',  include('budgets.urls')),
    path('reports/',  include('reports.urls')),
    path('',          include('core.dashboard_urls')),
]
```

Each app then has its own `urls.py`. For example, `core/urls.py`:

```
urlpatterns = [
    path('',              EntryListView.as_view(),   name='
       entry_list'),
    path('add/',          EntryCreateView.as_view(), name='
       entry_create'),
    path('<int:pk>/edit/',EntryUpdateView.as_view(), name='
       entry_update'),
    path('<int:pk>/del/', EntryDeleteView.as_view(), name='
       entry_delete'),
    path('categories/',   CategoryListView.as_view(),name='
       category_list'),
    ...
]
```

Chapter 5

Database Design

5.1 Models Overview

The application has five main models. All models filter data by the logged-in user, so one user never sees another user's data.

Model	Purpose
User	Custom user model extending Django's AbstractUser, adds unique email
Category	Income or expense category, belongs to a user
Tag	Freeform label that can be attached to multiple entries
Entry	A single income or expense transaction
Budget	A monthly spending limit for one category

Table 5.1: Application models

5.2 Entity-Relationship Diagram

er_diagram.png

Figure 5.1: Entity-Relationship Diagram

Chapter 6

System Features and Implementation

6.1 Authentication

User registration and login are handled using Django's built-in auth system. We created a custom `User` model in the `accounts` app that extends `AbstractUser` and adds a unique email field.

When a new user registers, a Django signal automatically runs and creates 14 default categories (9 expense, 5 income) for them. This is done through a post-save signal in `accounts/signals.py` that calls a utility function from `core/utils.py`.

6.2 CRUD Operations

All the main features (entries, categories, budgets) follow the same pattern using Django's Class-Based Views (CBVs). Here is how the entry CRUD works as an example:

```
class EntryListView(LoginRequiredMixin, ListView):
    model = Entry
    context_object_name = 'entries'
    paginate_by = 25

    def get_queryset(self):
        # Always filter by the logged-in user
        return Entry.objects.filter(user=self.request.user)

class EntryCreateView(LoginRequiredMixin, CreateView):
    model = Entry
    form_class = EntryForm
    success_url = reverse_lazy('entry_list')

    def form_valid(self, form):
        # Attach the current user before saving
        form.instance.user = self.request.user
        return super().form_valid(form)
```

```
class EntryDeleteView(LoginRequiredMixin, UserPassesTestMixin,
    DeleteView):
     model = Entry

     def test_func(self):
          # Make sure only the owner can delete
          return self.get_object().user == self.request.user
```

Listing 6.1: Entry views using Django CBVs

The same pattern is used for categories and budgets. Every queryset filters by `user=request.user` to make sure users can only see their own data.

6.3 Forms and Validation

We used Django's ModelForm for all input forms. The `EntryForm` does the following validation:

- Amount must be greater than zero
- Date cannot be in the future
- The selected category's type must match the entry type (e.g. an income entry cannot use an expense category)
- Tags are entered as comma-separated text and saved as M2M relationships

The `BudgetForm` handles a special case: the HTML `<input type="month">` widget sends `YYYY-MM` but Django's `DateField` expects `YYYY-MM-DD`. The form's `clean_month()` method adds `-01` to fix this.

6.4 Dashboard

The dashboard uses Django's ORM to calculate monthly totals:

```
total_income = Entry.objects.filter(
     user=user,
     date__year=today.year,
     date__month=today.month,
     entry_type='INCOME'
).aggregate(total=Sum('amount'))['total'] or 0
```

It also generates data for two charts: a 30-day daily trend chart and an expenses-by-category donut chart. These are passed to the template as JSON and rendered using Chart.js.

6.5 Reports and CSV Export

The reports page allows users to filter by time period (this week, this month, last month, this year, custom range). The `ReportView` calculates totals for the selected period and passes them to a Chart.js bar chart.

The CSV export uses Python's built-in `csv` module:

```
class ExportCSVView(LoginRequiredMixin, View):
    def get(self, request):
        response = HttpResponse(content_type='text/csv')
        response['Content-Disposition'] = 'attachment; filename="
           report.csv"'
        writer = csv.writer(response)
        writer.writerow(['Date', 'Type', 'Category', 'Amount', '
           Description'])
        for entry in entries:
            writer.writerow([entry.date, entry.entry_type,
                             entry.category.name, entry.amount,
                             entry.description])
        return response
```

6.6 Custom Template Tags

We created a custom template tag library in `core/templatetags/expenser_tags.py`. This allows us to use filters directly in Django templates:

- `currency` – formats a number as `Tk. 1,234.56`
- `percentage` – calculates percentage safely (avoids division by zero)
- `tag_pills` – renders coloured badge tags for each entry
- `category_color_hex` – returns a colour based on the category name for avatar backgrounds

Chapter 7

Team Contributions

Name	Student ID	Role
Shihab Shaharia	2023100010136	Person 1
Abid Ahmed Angkon	2023200010039	Person 2
Mahfuz Iqram Sunny	2023200010034	Person 3

7.1 Shihab Shaharia — Person 1

- Set up the Django project, folder structure, PostgreSQL database, and virtual environment configuration
- Designed all database models and wrote the initial migrations
- Set up Django admin with list displays and filters for all models
- Built the authentication system including the custom User model and registration view
- Wrote the post-save signal that seeds default categories for new users
- Built the dashboard view with ORM aggregations and chart data
- Created all base HTML templates and the overall page layout
- Managed the GitHub repository, reviewed pull requests, and resolved merge conflicts

7.2 Abid Ahmed Angkon — Person 2

- Implemented all Entry views (list, create, update, delete) using Django CBVs
- Built the EntryForm with field-level and cross-field validation
- Implemented all Category views and CategoryForm
- Built all Budget views with spending calculation logic

- Handled the BudgetForm month field coercion from YYYY-MM to a date
- Added entry filtering by type, category, tag, and date range
- Wrote the `process_recurring` management command
- Wired up the slide-over drawer for add/edit forms

7.3 Mahfuz Iqram Sunny — Person 3

- Built the ReportView with all date range presets and ORM aggregations
- Implemented the CSV export using Python's built-in csv module
- Wrote all custom template tags: `currency`, `percentage`, `tag_pills`, `category_color_hex`
- Set up Chart.js for the area trend chart, donut chart, and sparkline cards on the dashboard
- Designed and styled the reports page template
- Helped with UI adjustments across the entry list and category pages

Chapter 8

Application Screenshots

The following screenshots show the final working application running on the local development server.

8.1 Dashboard

The dashboard shows the monthly income, expense, and net balance stat cards with sparkline charts, a 30-day area trend chart, a donut chart for expenses by category, a recent entries table, and a budget progress panel on the right.

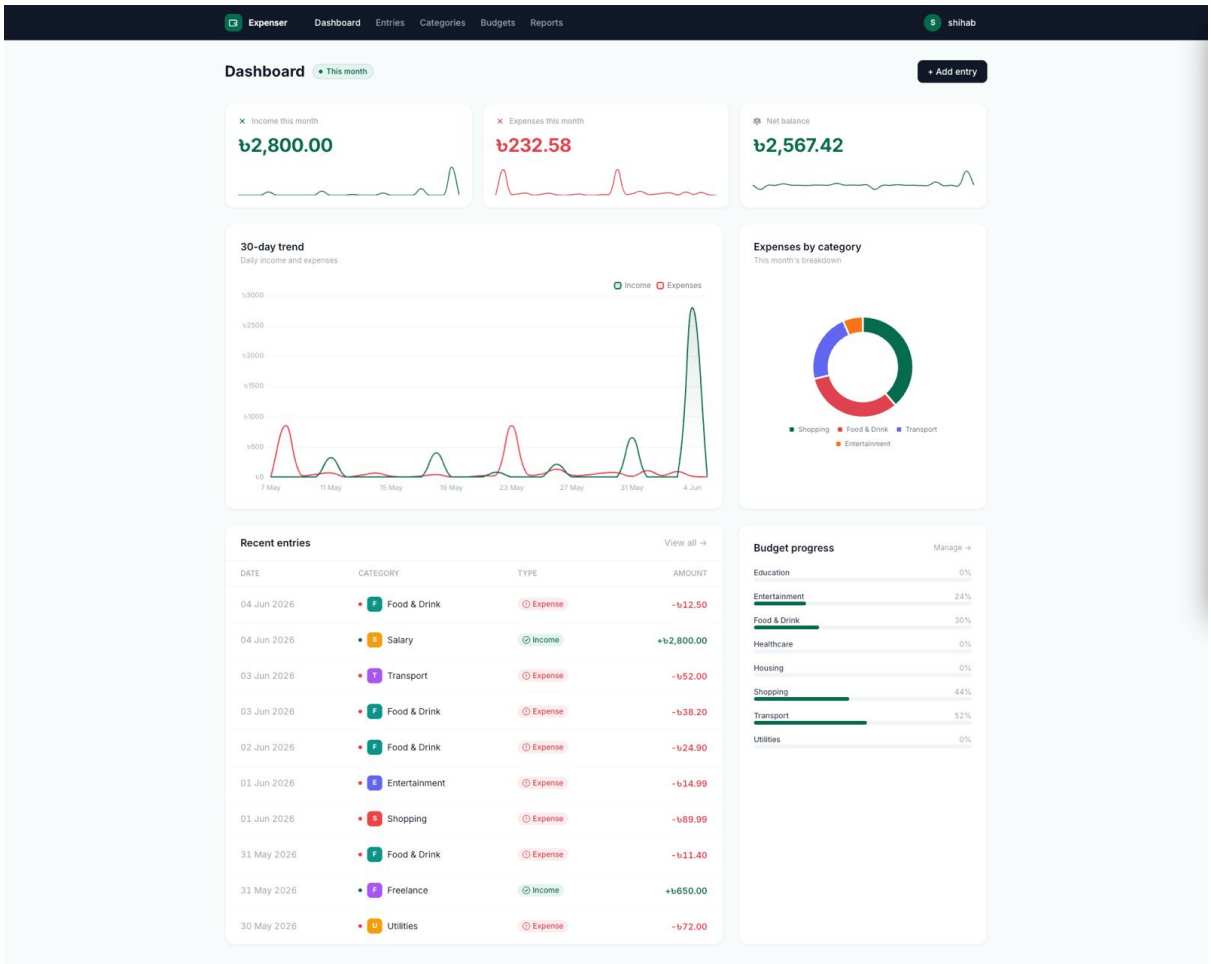

Figure 8.1: Dashboard page

8.2 Entries

The entries page lists all transactions with filtering by type, category, tag, and date range. Each row shows the date, type badge, category avatar, description, and amount coloured green for income and red for expense.

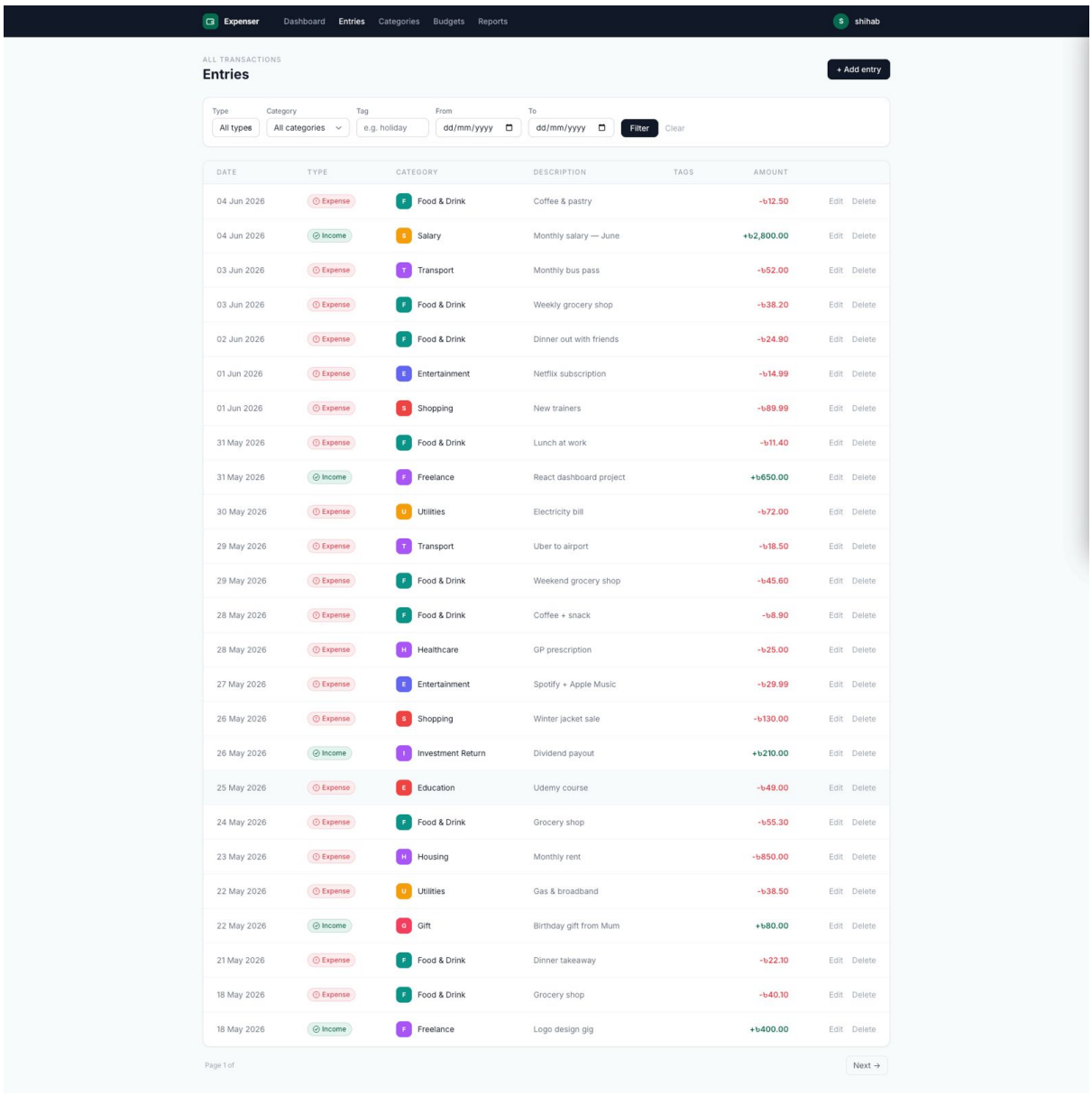

Figure 8.2: Entries list page with filters

8.3 Categories

The categories page splits all categories into Income and Expense sections, each with a coloured letter avatar, type badge, and Edit/Delete actions.

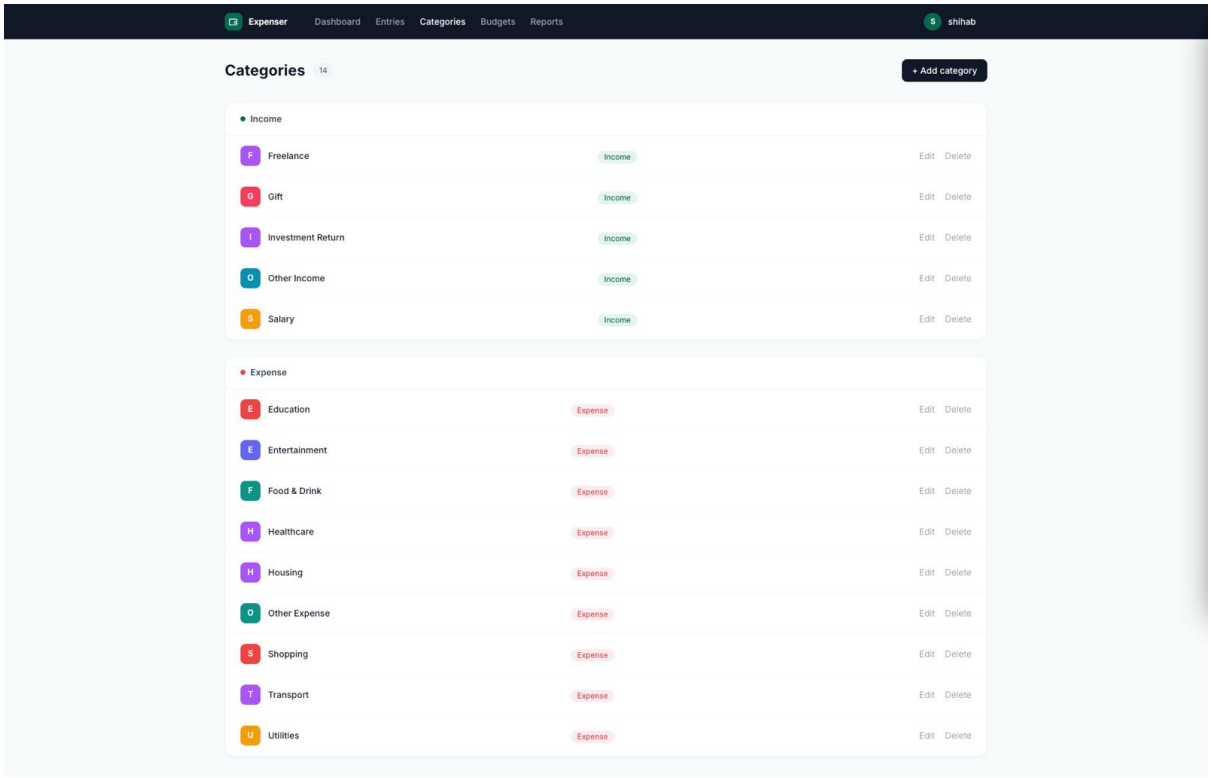

Figure 8.3: Categories management page

8.4 Budgets

The budgets page shows each monthly budget with a progress bar indicating how much of the limit has been spent. The colour changes from green to amber at 80% and red when the limit is exceeded.

8.5 Reports

The reports page allows selecting a date range preset and shows total income, expenses, and net balance for the period. A bar chart breaks down the top expense categories. Reports can be downloaded as a CSV file.

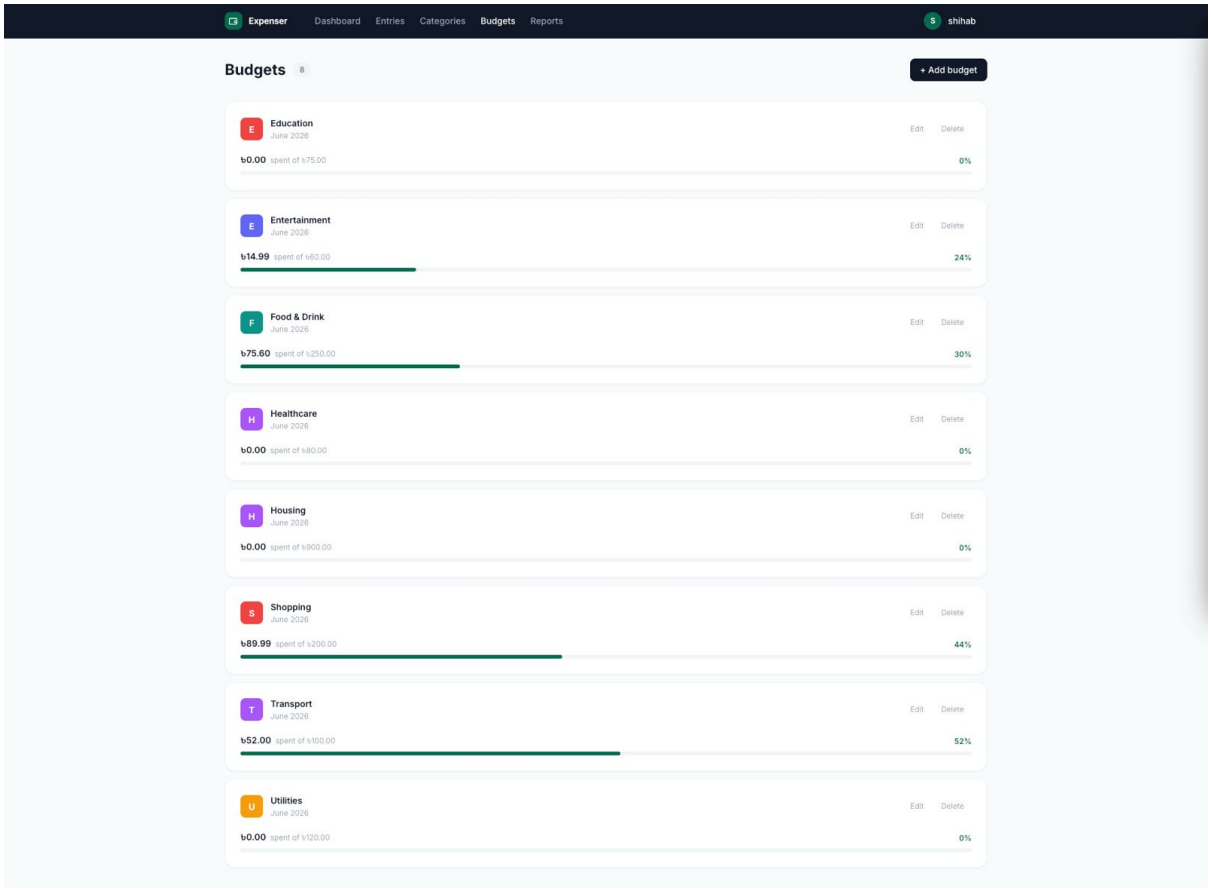

Figure 8.4: Budgets page with progress bars

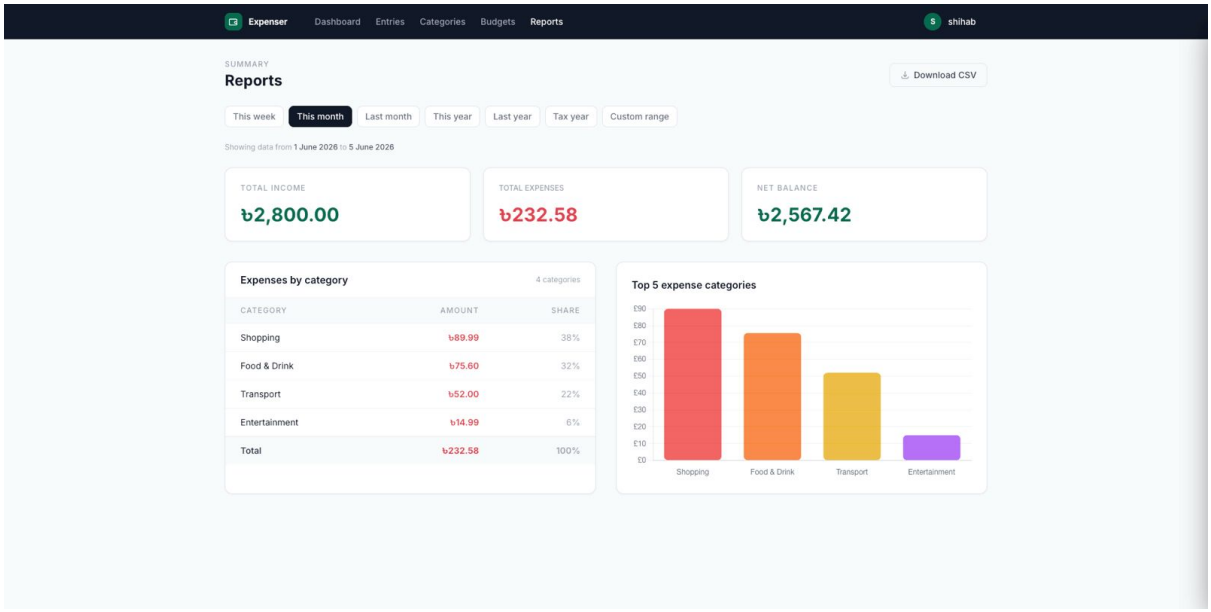

Figure 8.5: Reports page with date range selector and bar chart

Chapter 9

Future Work

The current version of Expenser covers all the core requirements of the project. However, there are several features we discussed during development that we did not have time to implement. These would be good additions in a future version:

- **PDF report export** — Currently reports can only be downloaded as CSV. Adding PDF export using a library like WeasyPrint would make the reports more presentable and easier to share.
- **Email notifications** — The budget warning system currently only shows alerts on the page. It would be more useful to send an email to the user when they reach 80% or exceed their budget limit for a category.
- **Automated recurring entry processing** — The `process_recurring` management command exists but has to be run manually. Scheduling it with a cron job on the Linux server would make recurring entries fully automatic.
- **Mobile responsive layout** — The application works on desktop but the layout breaks on smaller screens. Improving the responsive design with Tailwind's mobile breakpoints would make it usable on phones.
- **User profile page** — Currently users cannot change their username, email, or password from within the app. Adding a profile settings page would make account management easier.
- **Multi-currency support** — Right now everything is in BDT. Allowing users to select their currency and optionally converting between currencies using an external API would make the app more internationally useful.
- **Automated tests** — We did all our testing manually. Writing a proper test suite using `pytest-django` would make it easier to catch regressions when adding new features in the future.

Chapter 10

Conclusion

We built Expenser as a working personal finance tracker using Django and PostgreSQL. The project helped us understand how to structure a real Django application with multiple apps, how to use class-based views properly, how forms and validation work, and how to use signals and custom management commands.

Working as a team through GitHub taught us how to manage branches, review each other's code, and merge work without breaking things. We ran into a few issues along the way, like resolving merge conflicts and debugging template rendering problems, but we sorted them out as we went.

The application covers all the core requirements: user authentication, transaction tracking, category and budget management, financial reports, and CSV export. It runs on a Linux virtual machine and the full setup process is documented in this report. The project gave us practical experience in team-based software development using Agile, which we think is the most valuable thing we took from this course.

Git Repository: https://github.com/shihabshaharia/expenser

Access: Public repository — no credentials required

Bibliography

[1] Django Software Foundation, *Django Documentation*, version 5.2. https://docs.djangoproject.com

[2] PostgreSQL Global Development Group, *PostgreSQL Documentation*. https://www.postgresql.org/docs

[3] Tailwind Labs, *Tailwind CSS Docs*. https://tailwindcss.com/docs

[4] Chart.js Contributors, *Chart.js Docs*. https://www.chartjs.org/docs